The Memory Hierarchy

Youngjae Kim (Ph.D.)
Sogang University
Linux 운영체제 및 응용 (GITF315-01)

Today

- **Storage technologies and trends**
- **Locality of reference**
- **Caching in the memory hierarchy**

DRAM은 싸고 효율적임.
SRAM은 비싸지만 빠름.

DRAM :
- 회로가 작고 간단한데.. 전력이 조금씩 소실되기 때문에 계속해서 refresh를 해줘야 함. 해당 문제 때문에 SRAM에 비해 느림.

SRAM :
- 회로가 복잡하고 cpu에 캐시처럼 붙기 때문에 비쌈.
- switch를 사용하기 때문에 1/0이 명확하고, 전력 소실이 없어 약 10배정도 빠름.

Random-Access Memory (RAM)

Memory 아무곳에나 엑세스해도 동일하게 사용할 수 있다.

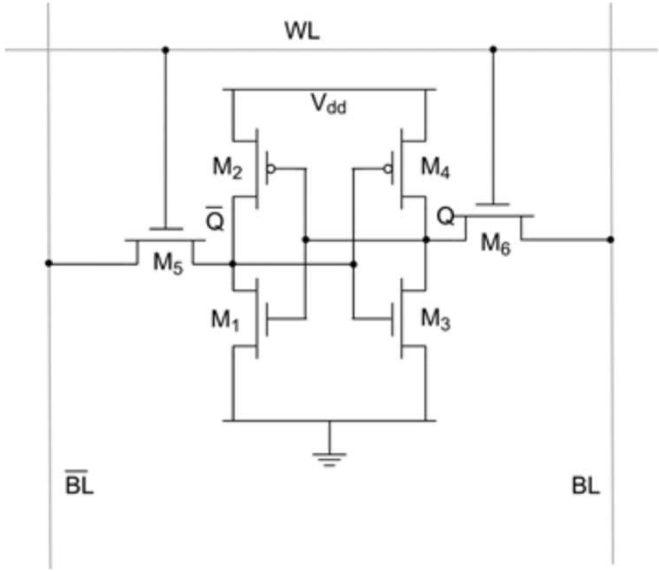
- **Key features**
  - RAM is traditionally packaged as a chip.
  - Basic storage unit is normally a cell (one bit per cell).
  - Multiple RAM chips form a memory.

- **RAM comes in two varieties:**
  - SRAM (Static RAM)
  - DRAM (Dynamic RAM)

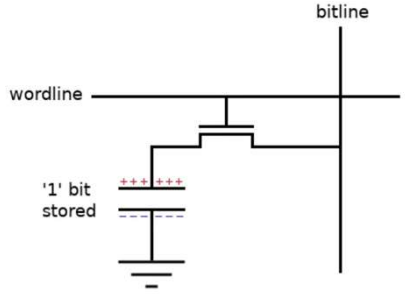
SRAM vs DRAM Summary

SRAM은 cpu에 캐시처럼 붙어 있다.

Feature	SRAM	DRAM
Cell Structure	6T (Six Transistors)	1T1C (One Transistor + One Capacitor)
Bit Storage Method	Cross-coupled latch formed by transistors	Charge stored in a capacitor
Transistors per Bit	6 transistors	1 transistor
Capacitor Requirement	Not required	1 capacitor required
Cell Area	Large cell size	Much smaller cell size
Memory Density	**Low density**	**High density**
Cost per Bit	High	Low
Typical Usage	CPU caches (L1, L2, L3), register files	Main memory (system DRAM)



<SRAM with 6 transistors>



<DRAM with 1 transistor and 1 capacitor>

SRAM vs DRAM Summary

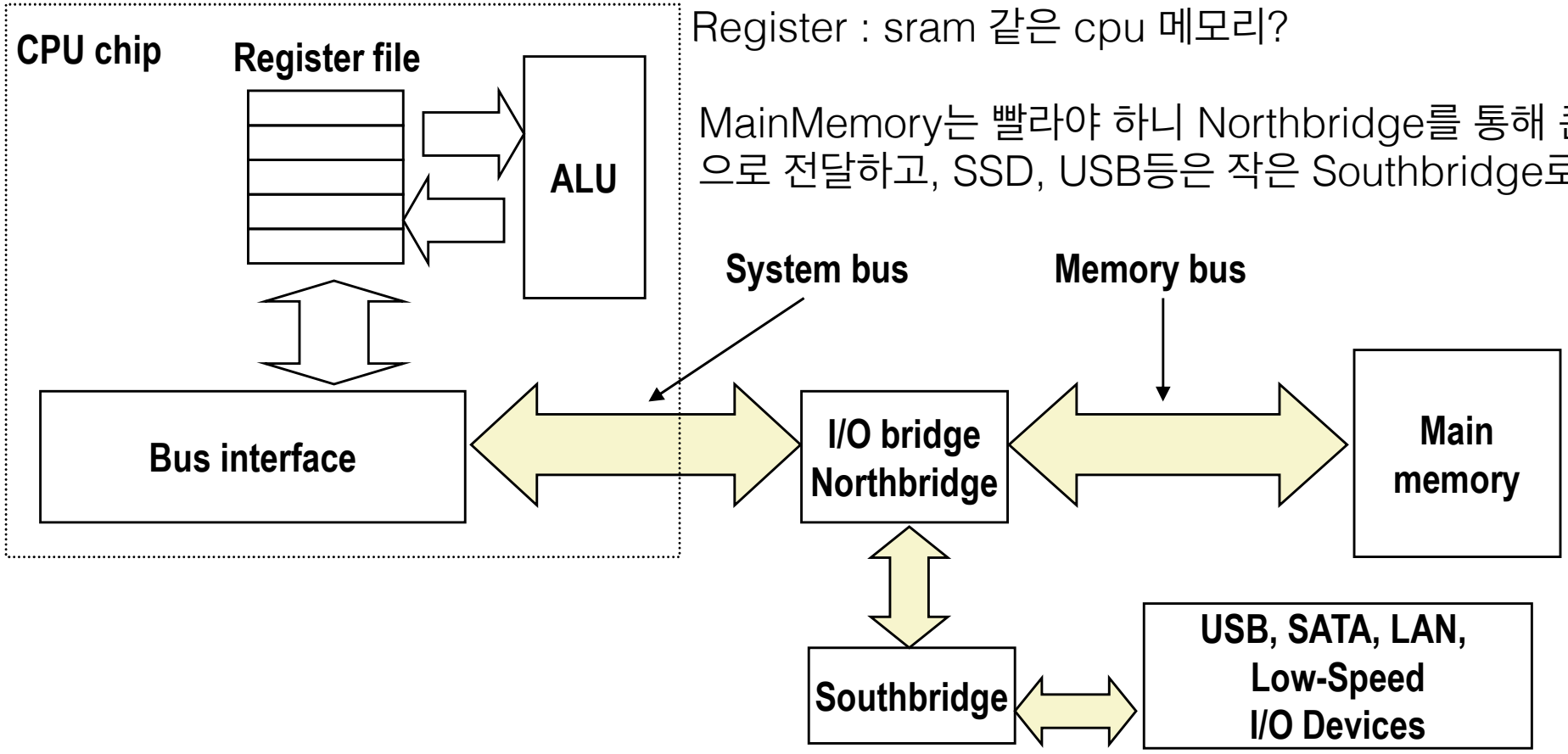
- **Access Time**
  - SRAM: Fast, because the stored state can be read directly from the latch.
  - DRAM: Slow, because the capacitor charge must be sensed and amplified.
- **Refresh**
  - SRAM: Not required, because the latch maintains its state as long as power is supplied
  - DRAM: Required, because the stored charge in the capacitor gradually leaks.

	Trans. per bit	Access time	Needs refresh?	Needs EDC?	Cost	Applications
SRAM	4 or 6	1X	No	Maybe	100x	Cache memories
DRAM	1	10X	Yes	Yes	1X	Main memories, frame buffers

*EDC: Error Detectin Code

Bus Structure Connecting CPU and Memory

- A **bus** is a collection of parallel wires that carry address, data, and control signals.
- Buses are typically shared by multiple devices.



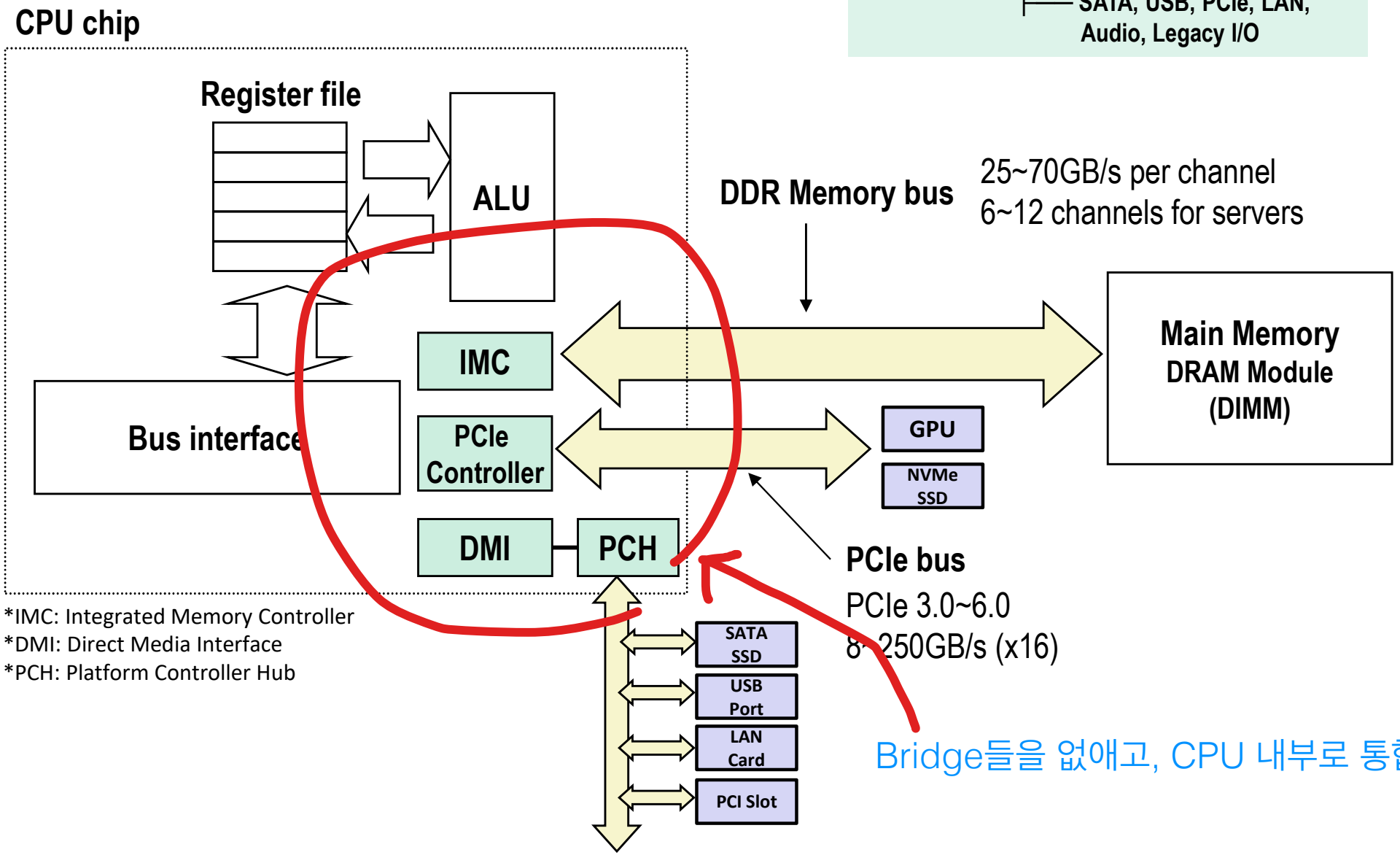
Register : sram 같은 cpu 메모리?

MainMemory는 빨라야 하니 Northbridge를 통해 큰 대역폭으로 전달하고, SSD, USB등은 작은 Southbridge로 사용됨

New

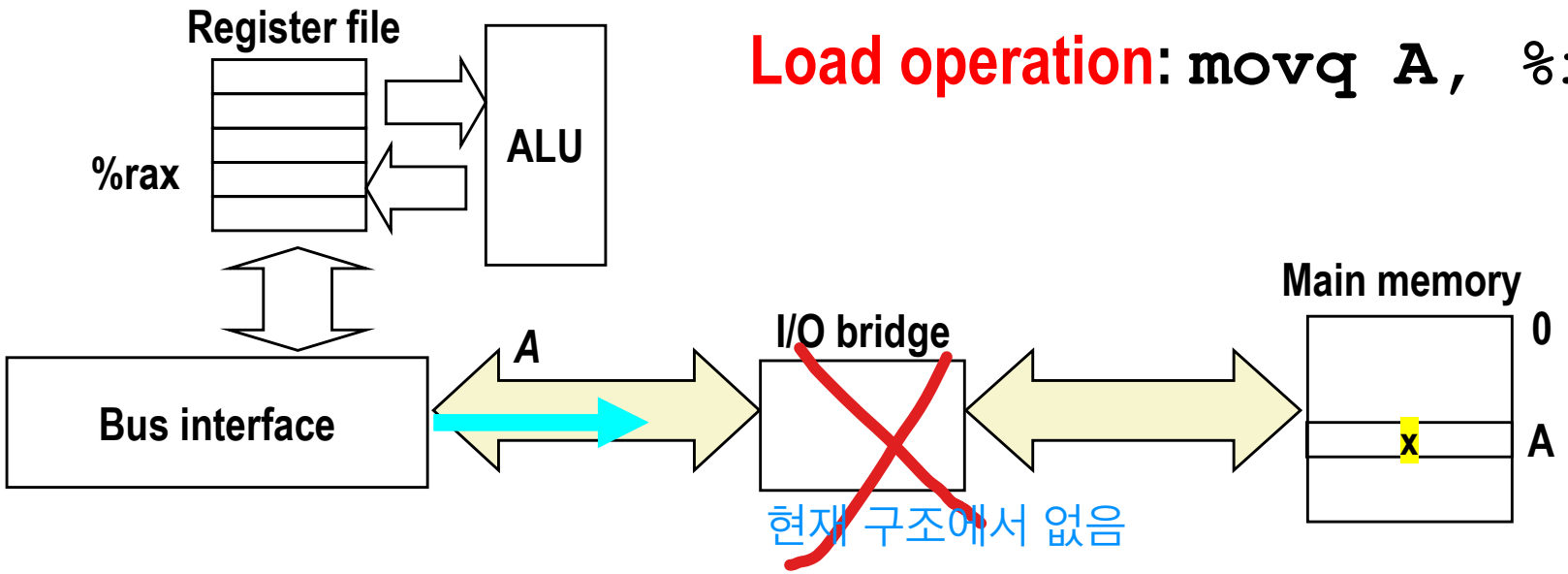
Modern Bus Structure

[CPU]
- DDR Memory Controller → DRAM
- PCIe Controller → GPU / NVMe SSD
- DMI → PCH
  - SATA, USB, PCIe, LAN, Audio, Legacy I/O



Bridge들을 없애고, CPU 내부로 통합됨.

Memory Read Transaction (1)

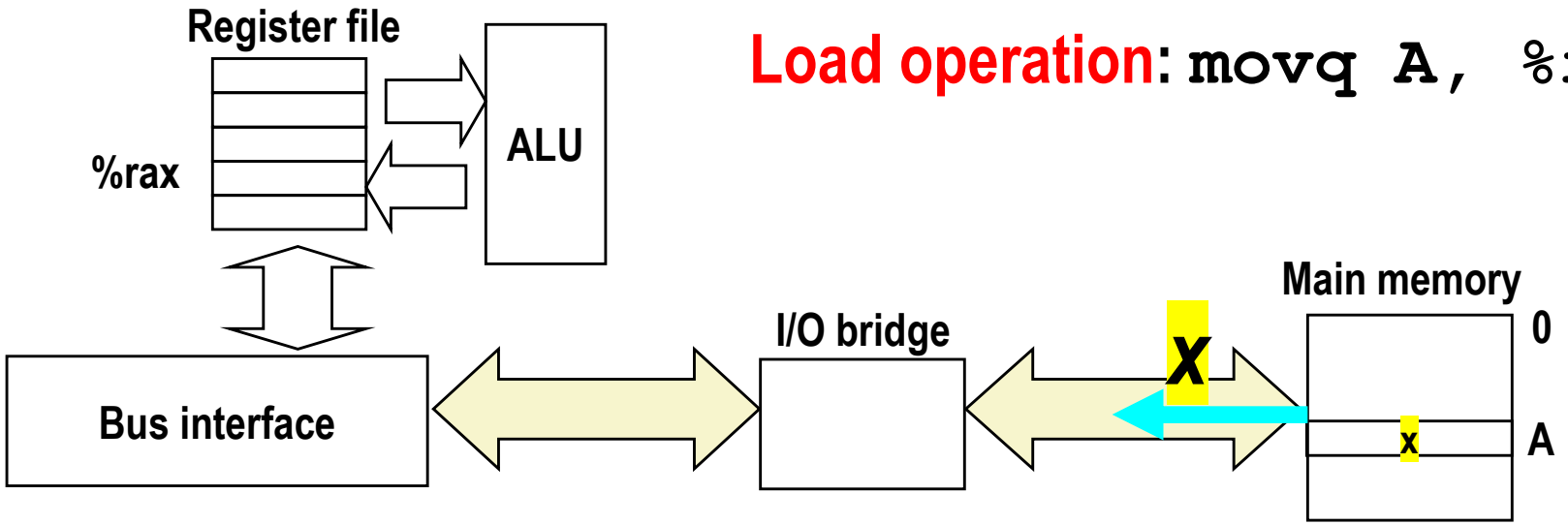


① **CPU places address A on the memory bus.**

② Main memory reads A from the memory bus, retrieves word x, and places it on the bus.

③ CPU read word x from the bus and copies it into register %rax.

Memory Read Transaction (2)

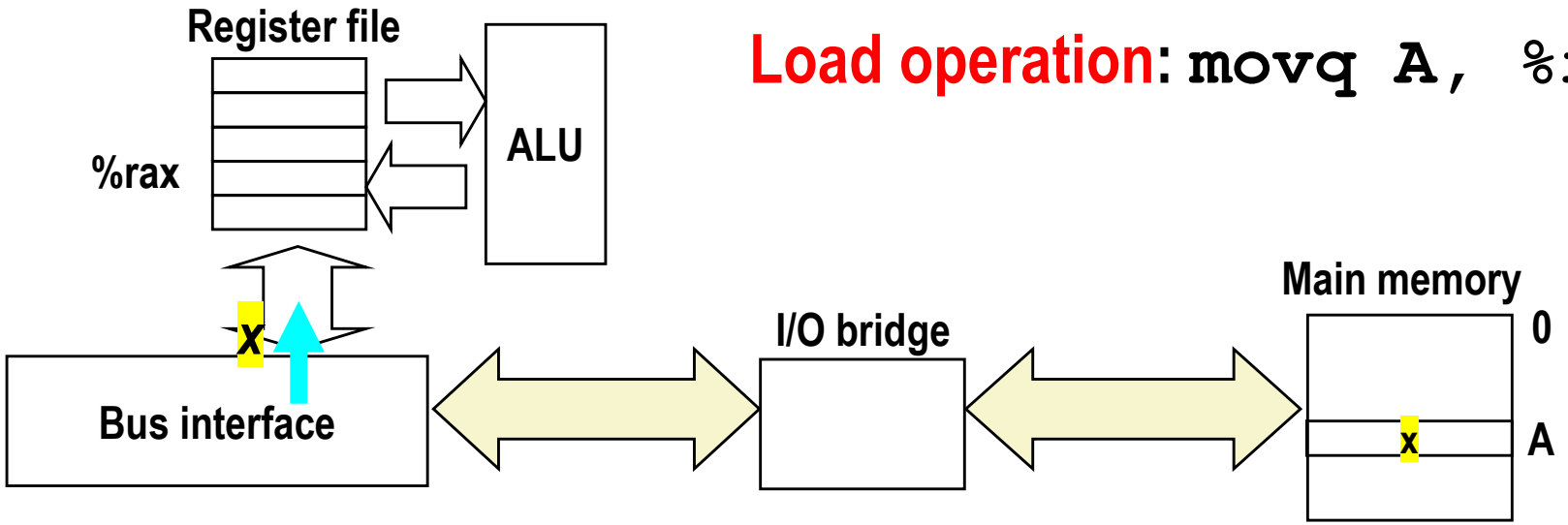


① CPU places address A on the memory bus.

② Main memory reads A from the memory bus, retrieves word x, and places it on the bus.

③ CPU read word x from the bus and copies it into register %rax.

Memory Read Transaction (3)



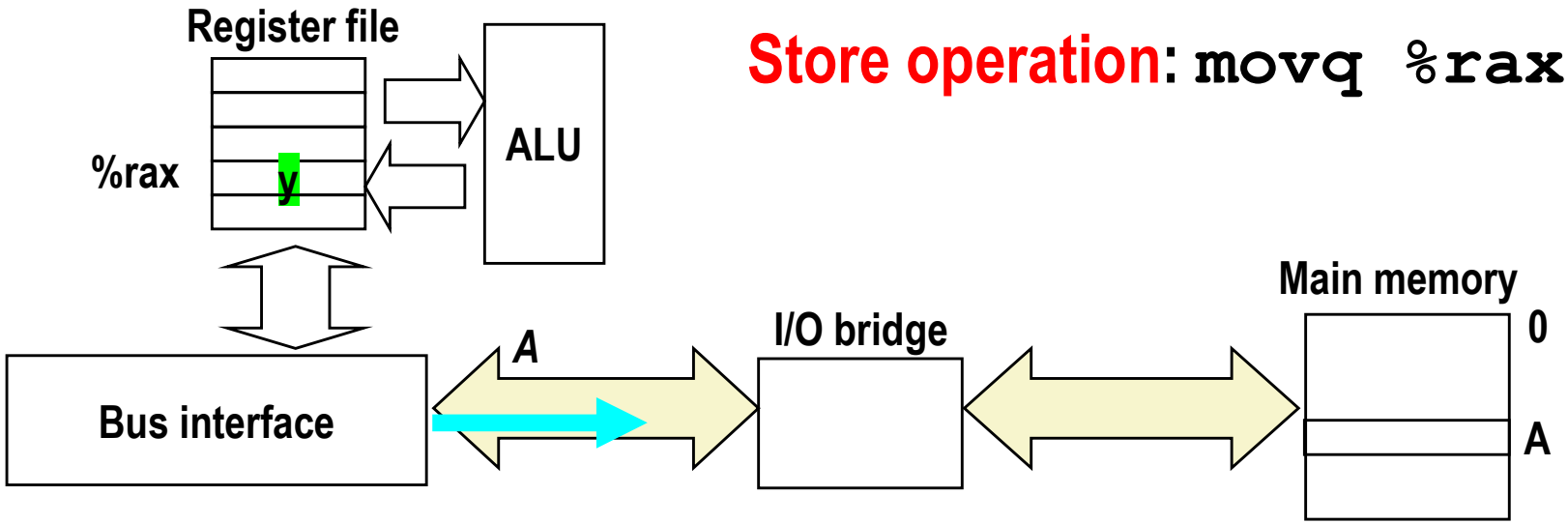
Load operation: `movq A, %rax`

① CPU places address A on the memory bus.

② Main memory reads A from the memory bus, retrieves word x, and places it on the bus.

③ CPU read word x from the bus and copies it into register %rax.

Memory Write Transaction (1)

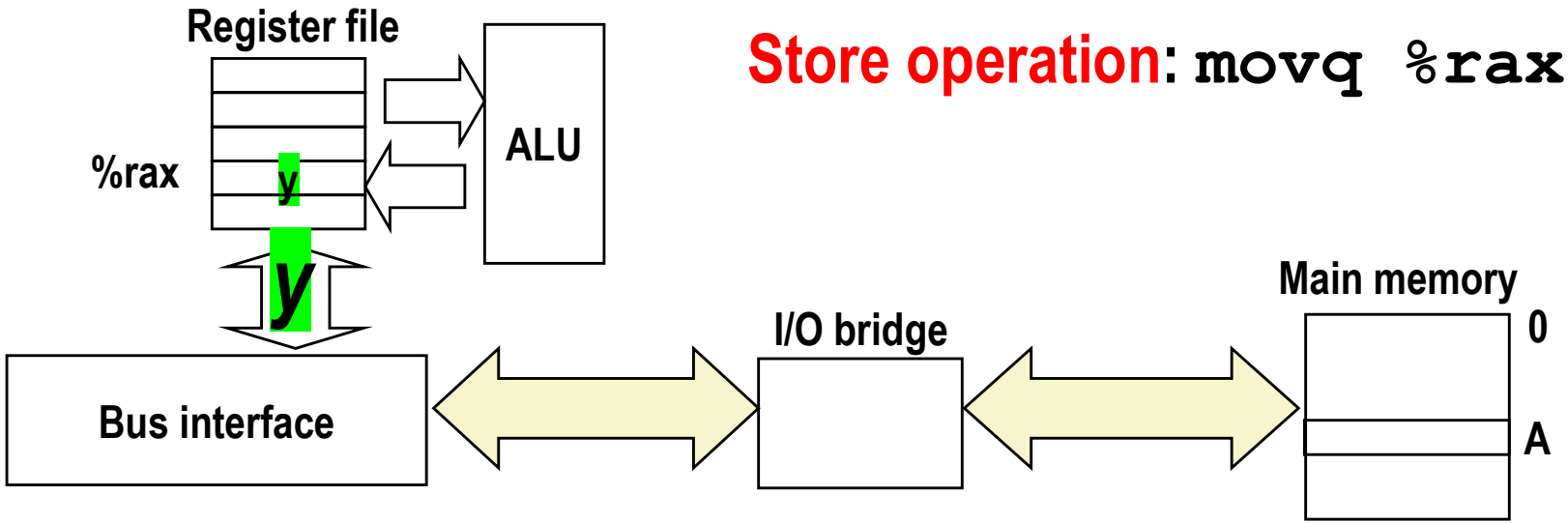


① CPU places address A on bus. Main memory reads it and waits for the corresponding data word to arrive.

② CPU places data word y on the bus.

③ Main memory reads data word y from the bus and stores it at address A.

Memory Write Transaction (2)

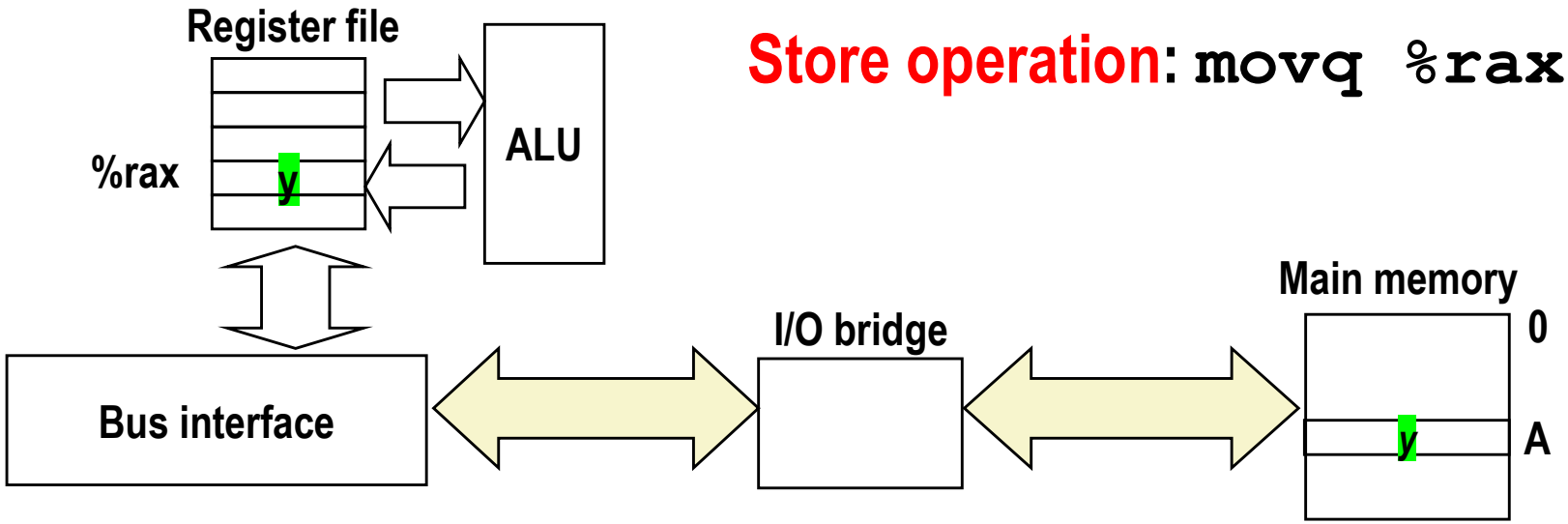


① CPU places address A on bus. Main memory reads it and waits for the corresponding data word to arrive.

② CPU places data word *y* on the bus.

③ Main memory reads data word y from the bus and stores it at address A.

Memory Write Transaction (3)



① CPU places address A on bus. Main memory reads it and waits for the corresponding data word to arrive.

② CPU places data word *y* on the bus.

③ Main memory reads data word *y* from the bus and stores it at address A.

What's Inside A Disk Drive?

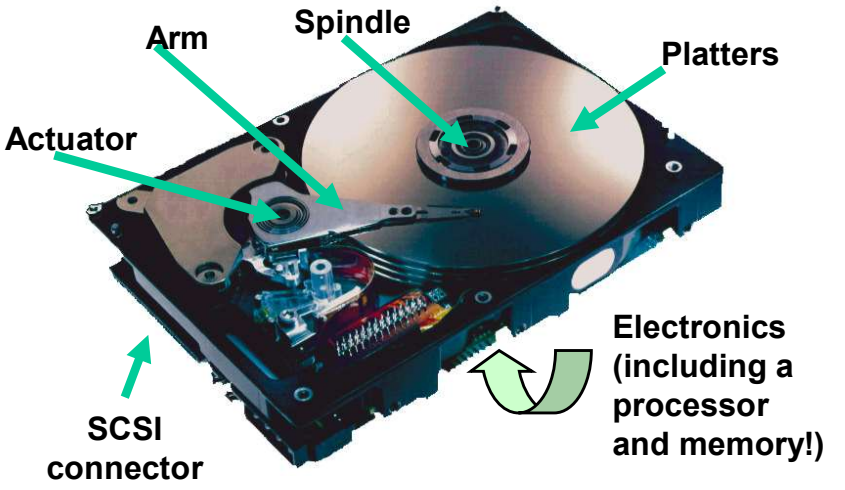
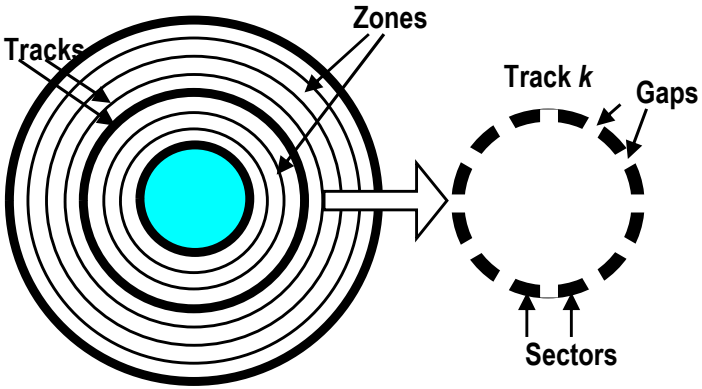
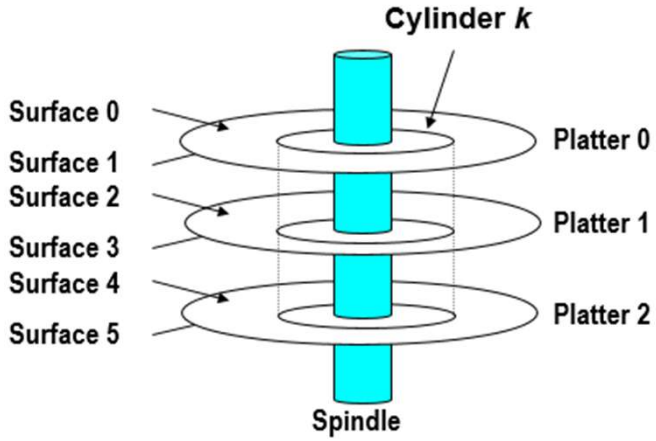


Image courtesy of Seagate Technology

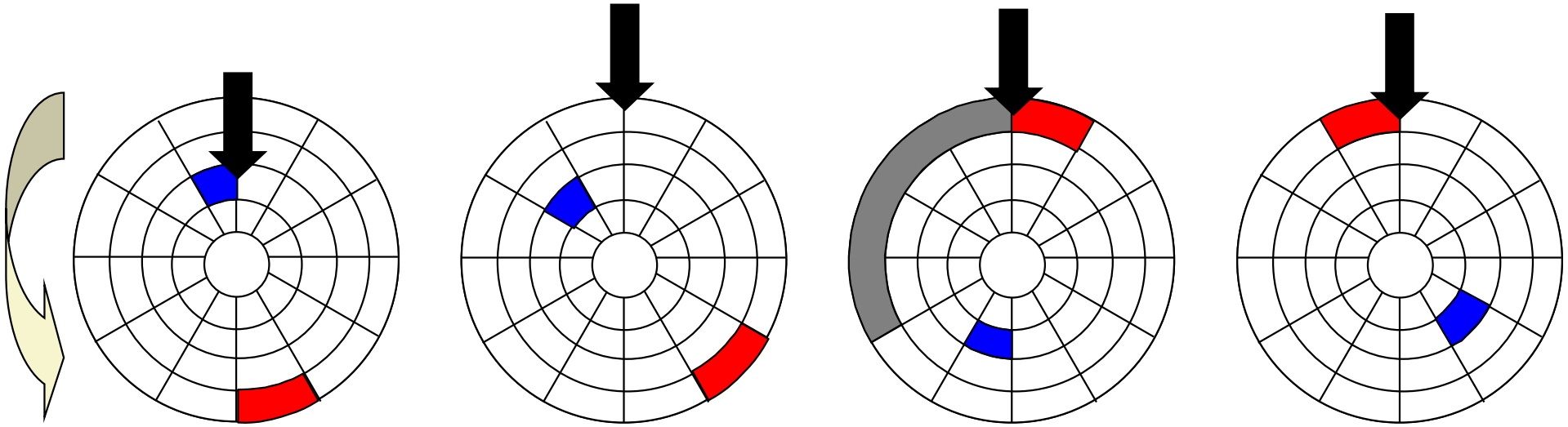




- **Capacity: maximum number of bits that can be stored.**
  - Vendors express capacity in units of gigabytes (GB), where 1 GB = 10^9 Bytes.
- **Capacity is determined by these technology factors:**
  - Recording density (bits/in): number of bits that can be squeezed into a 1 inch segment of a track.
  - Track density (tracks/in): number of tracks that can be squeezed into a 1 inch radial segment.
  - Areal density (bits/in2): product of recording and track density.

Capacity = (# bytes/sector) x (avg. # sectors/track) x (# tracks/surface) x (# surfaces/platter) x (# platters/disk)

Disk Access – Service Time Components



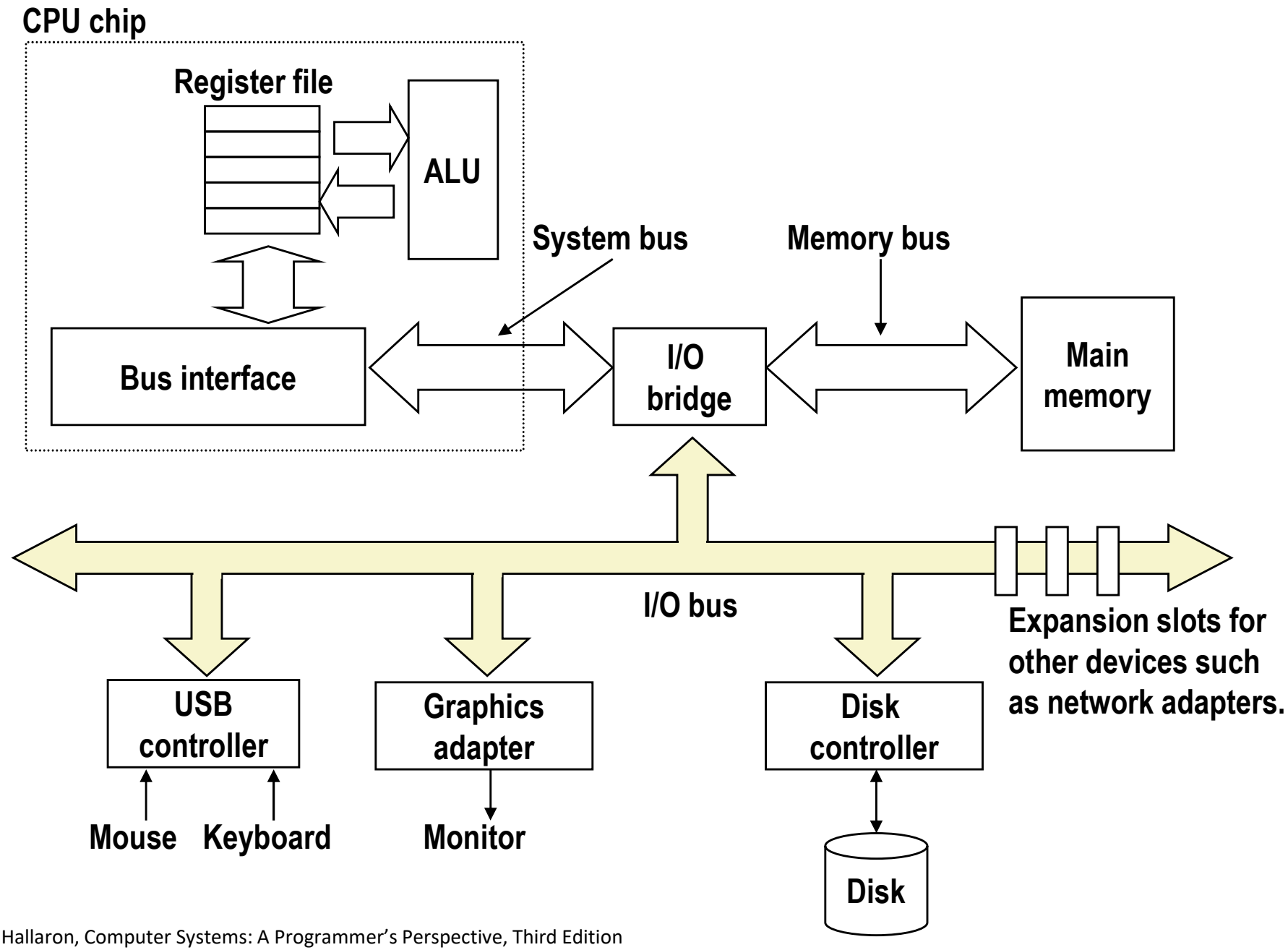
Disk Access Time

- **Average time to access some target sector approximated by :**
  - Taccess = Tavg seek + Tavg rotation + Tavg transfer
- **Seek time (Tavg seek)**
  - Time to position heads over cylinder containing target sector.
  - Typical Tavg seek is 3—9 ms
- **Rotational latency (Tavg rotation)**
  - Time waiting for first bit of target sector to pass under r/w head.
  - Tavg rotation = 1/2 x 1/RPMs x 60 sec/1 min
  - Typical Tavg rotation = 7200 RPMs
- **Transfer time (Tavg transfer)**
  - Time to read the bits in the target sector.
  - Tavg transfer = 1/RPM x 1/(avg # sectors/track) x 60 secs/1 min.

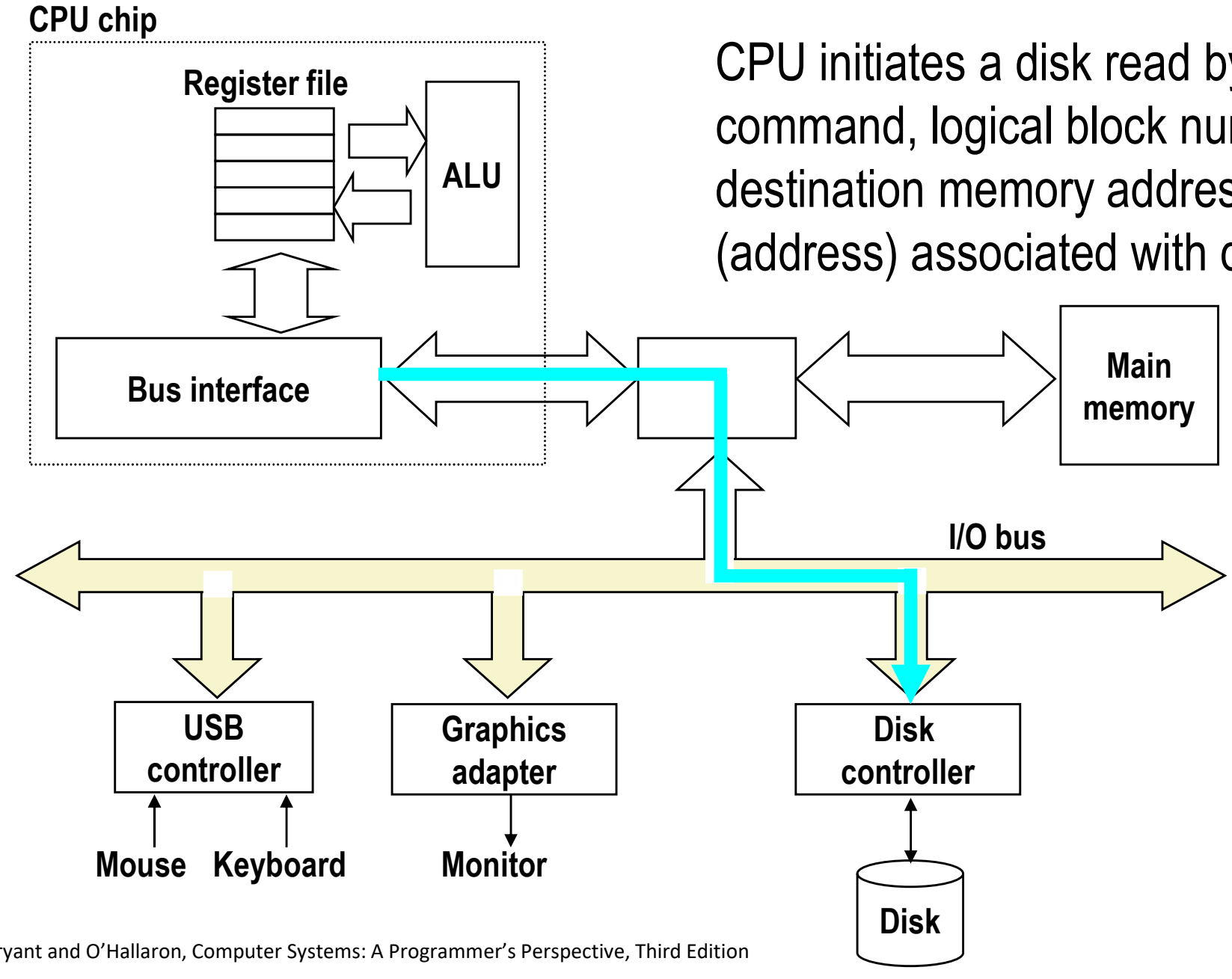
Logical Disk Blocks

- **Modern disks present a simpler abstract view of the complex sector geometry:**
  - The set of available sectors is modeled as a sequence of b-sized logical blocks (0, 1, 2, ...)
- **Mapping between logical blocks and actual (physical) sectors**
  - Maintained by hardware/firmware device called disk controller.
  - Converts requests for logical blocks into (surface,track,sector) triples.
- **Allows controller to set aside spare cylinders for each zone.**
  - Accounts for the difference in “formatted capacity” and “maximum capacity”.

I/O Bus

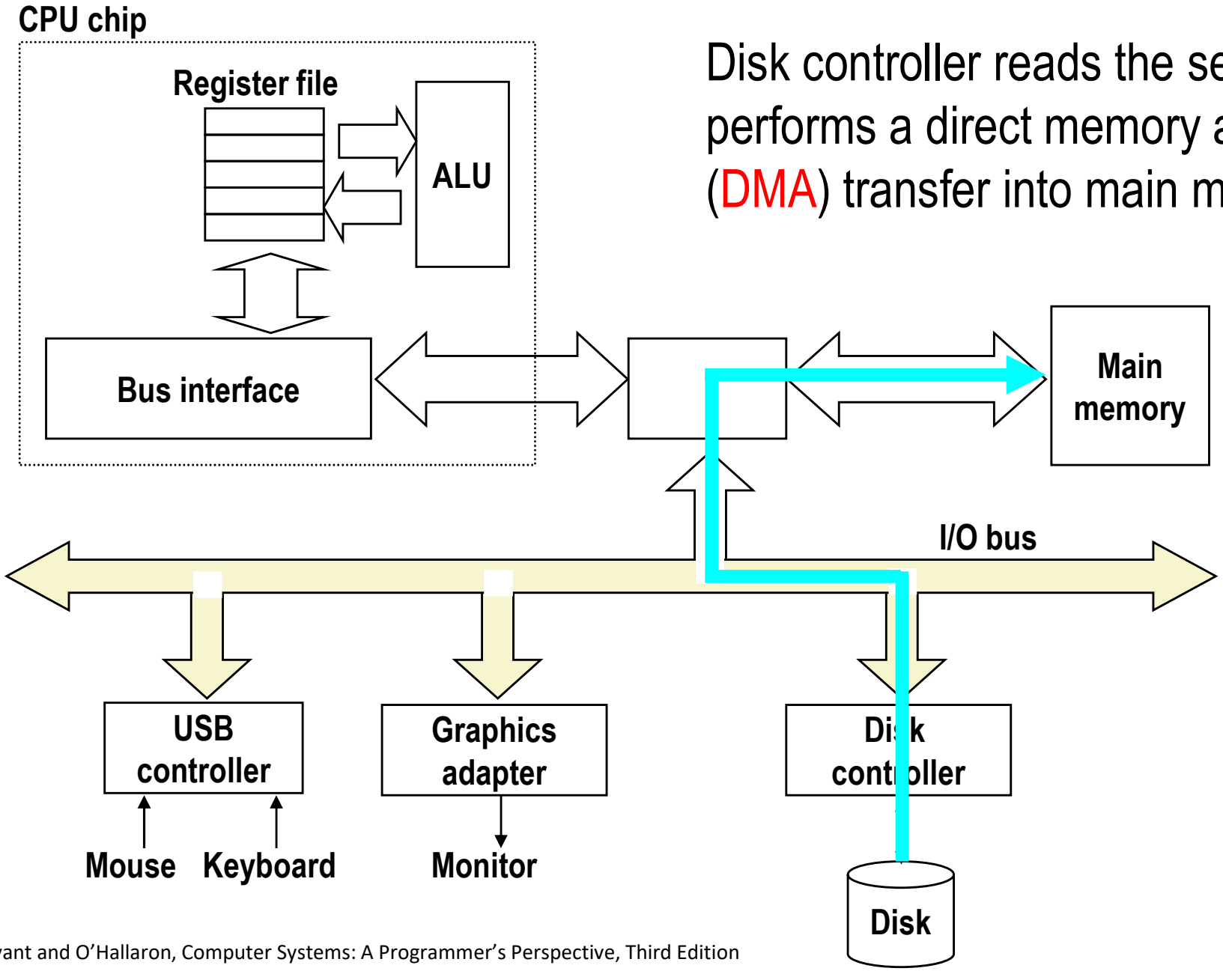

CPU chip
Register file
ALU
System bus
Memory bus
Bus interface
I/O bridge
Main memory
I/O bus
Expansion slots for other devices such as network adapters.
USB controller
Graphics adapter
Disk controller
Mouse
Keyboard
Monitor
Disk

Reading a Disk Sector (1)



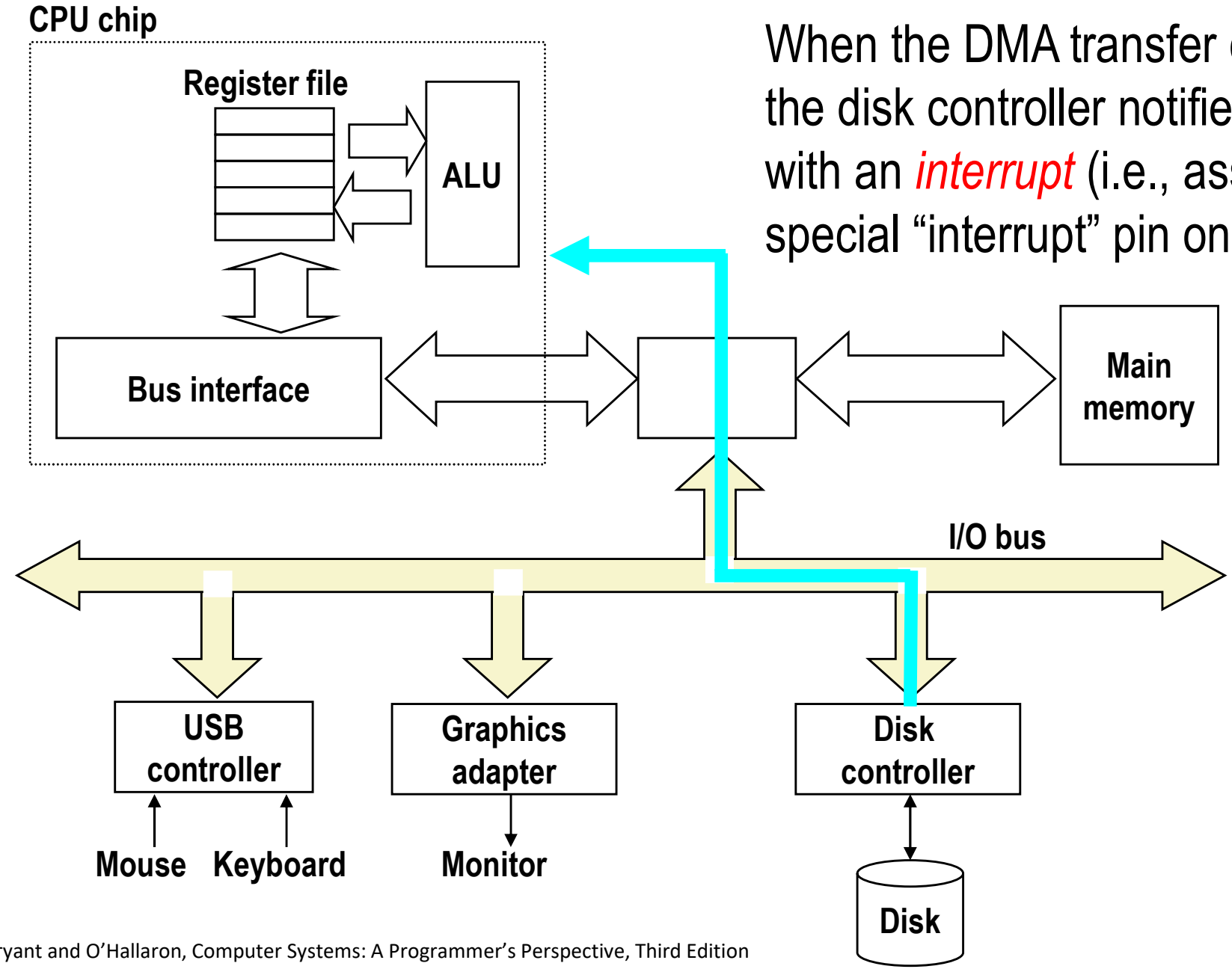
CPU initiates a disk read by writing a command, logical block number, and destination memory address to a port (address) associated with disk controller.

Reading a Disk Sector (2)



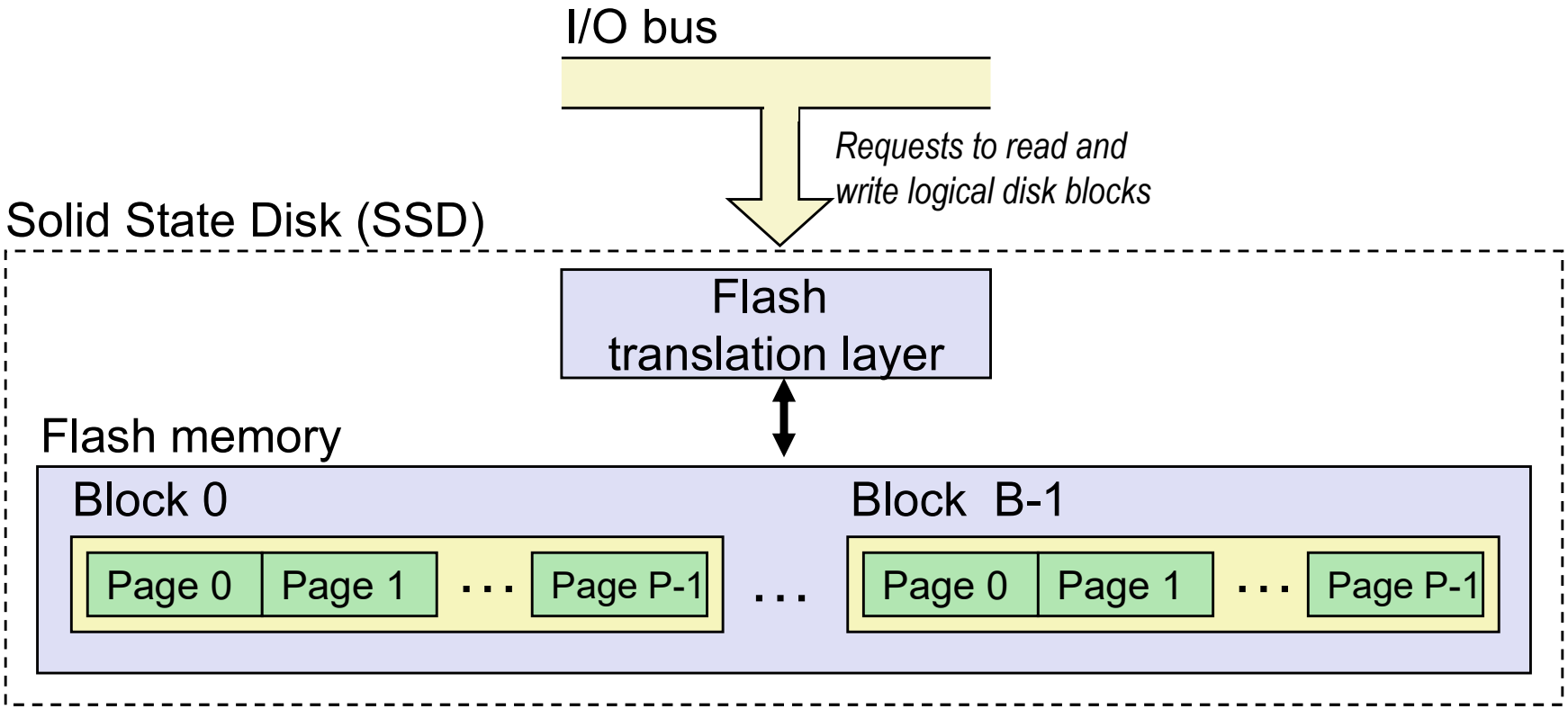
Disk controller reads the sector and performs a direct memory access (DMA) transfer into main memory.

Reading a Disk Sector (3)



When the DMA transfer completes, the disk controller notifies the CPU with an *interrupt* (i.e., asserts a special "interrupt" pin on the CPU)

Solid State Disks (SSDs)



- **Pages: 4KB to 16KB, Blocks: 256 to 1024 pages**
- **Data read/written in units of pages.**
- **Page can be written only after its block has been erased**
- **A block wears out after about 100,000 repeated writes.**

SSD Performance Characteristics

Sequential read tput	**13,500 MB/s**	**Sequential write tput**	**11,500 MB/s**
Random read tput	**1,600 K IOPS**	**Random write tput**	**1,500 K IOPS**
Avg seq read time	**~20 us**	**Avg seq write time**	**~25 us**

- **Sequential access faster than random access**
  - Common theme in the memory hierarchy
- **Random writes are somewhat slower**
  - Erasing a block takes a long time (~1 ms)
  - Modifying a block page requires all other pages to be copied to new block
  - In earlier SSDs, the read/write gap was much larger.

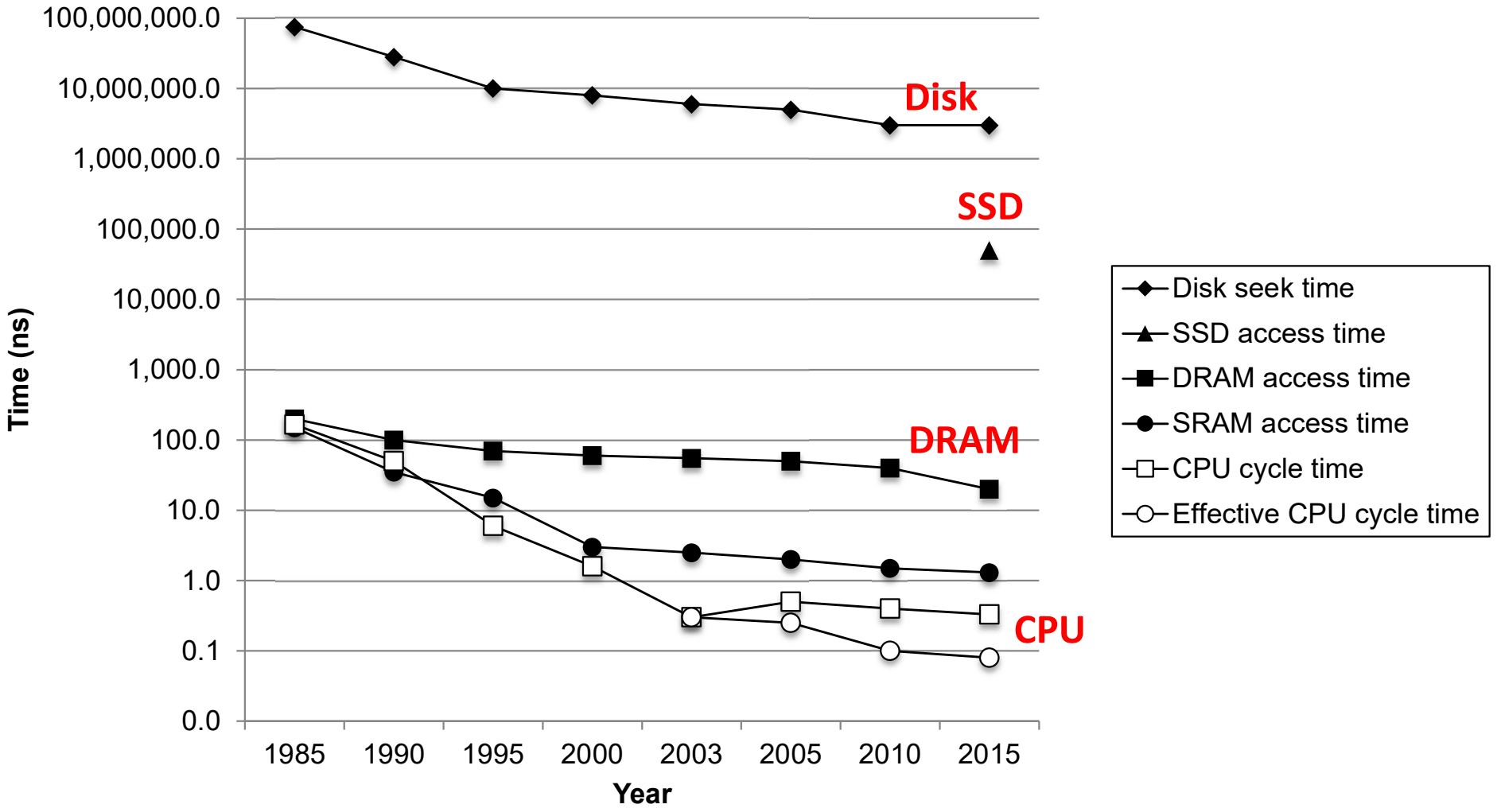
Source: SK hynix Platinum P51 (PCIe 5.0 NVMe SSD) product specification.

SSD Tradeoffs vs Rotating Disks

- **Advantages**
  - No moving parts → faster, less power, more rugged
- **Disadvantages**
  - Have the potential to wear out
    - Mitigated by “wear leveling logic” in flash translation layer
    - E.g. Intel SSD 730 guarantees 128 petabyte (128×10^{15} bytes) of writes before they wear out
  - In 2025, consumer-SSDs cost around US$0.079 per GB, while typical 3.5” internal HDDs cost about US$0.011–0.013 per GB.
  - SSDs **cost about six to seven times more** per byte than HDDs.

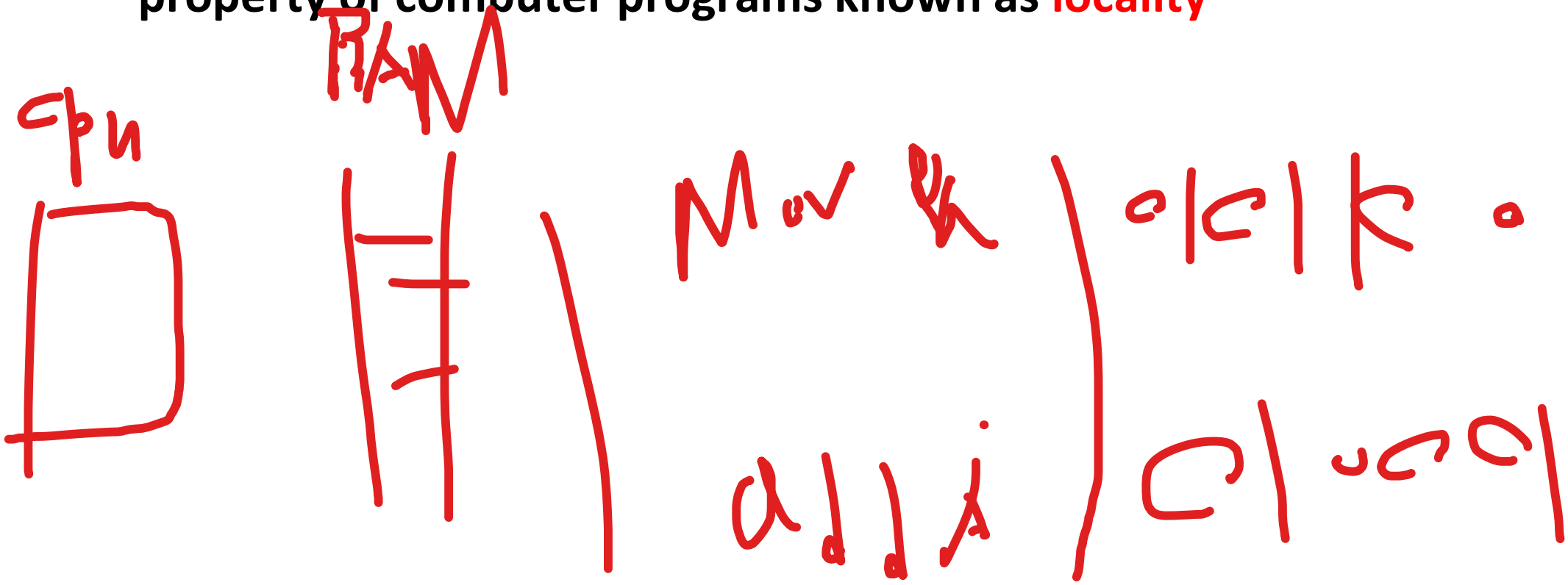
The CPU-Memory Gap

The gap widens between DRAM, disk, and CPU speeds.



Locality to the Rescue!

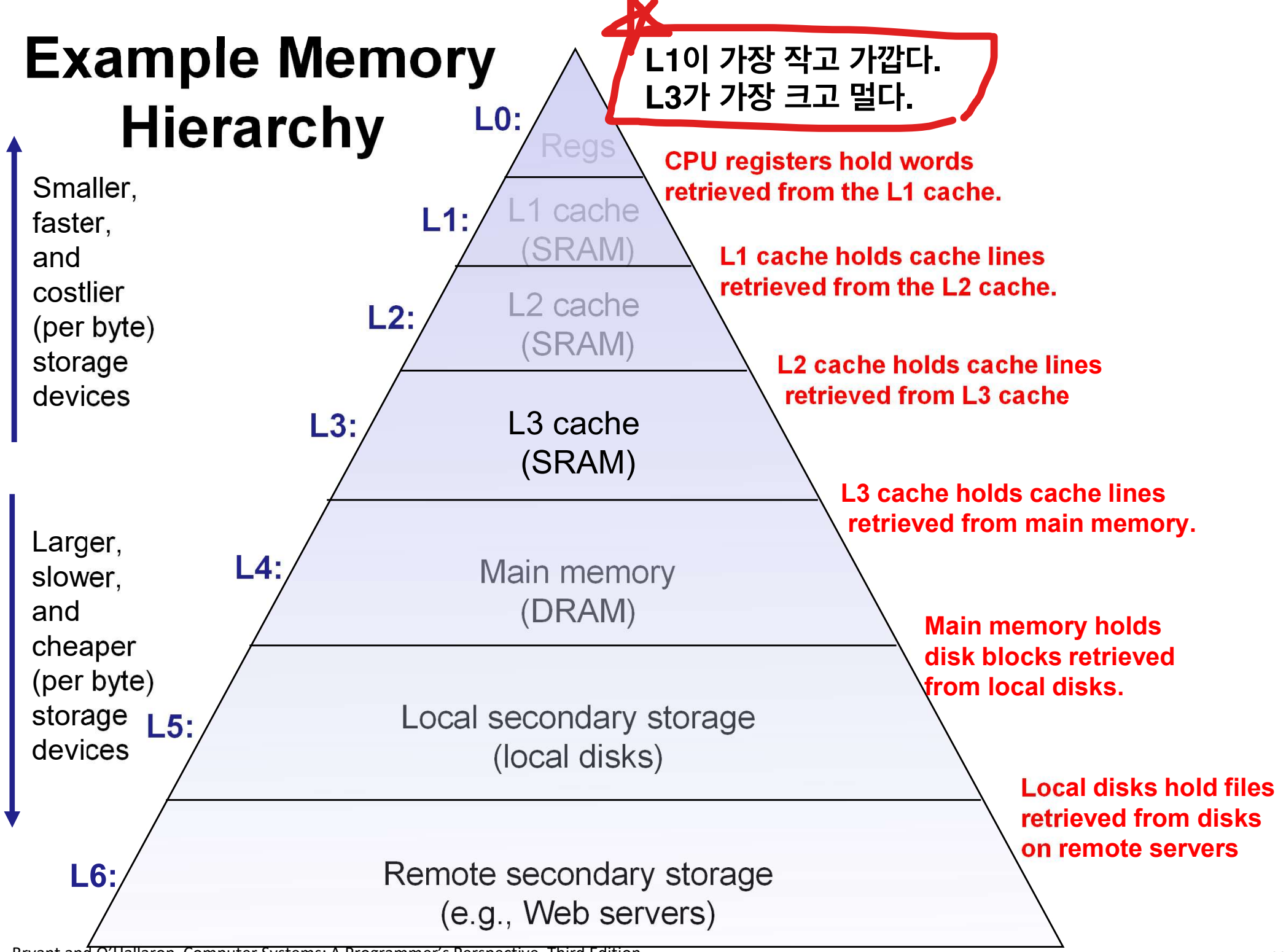
The key to bridging this CPU-Memory gap is a fundamental property of computer programs known as locality



Today

- **Cache memory organization and operation**
- **Performance impact of caches**
  - The memory mountain
  - Rearranging loops to improve spatial locality
  - Using blocking to improve temporal locality

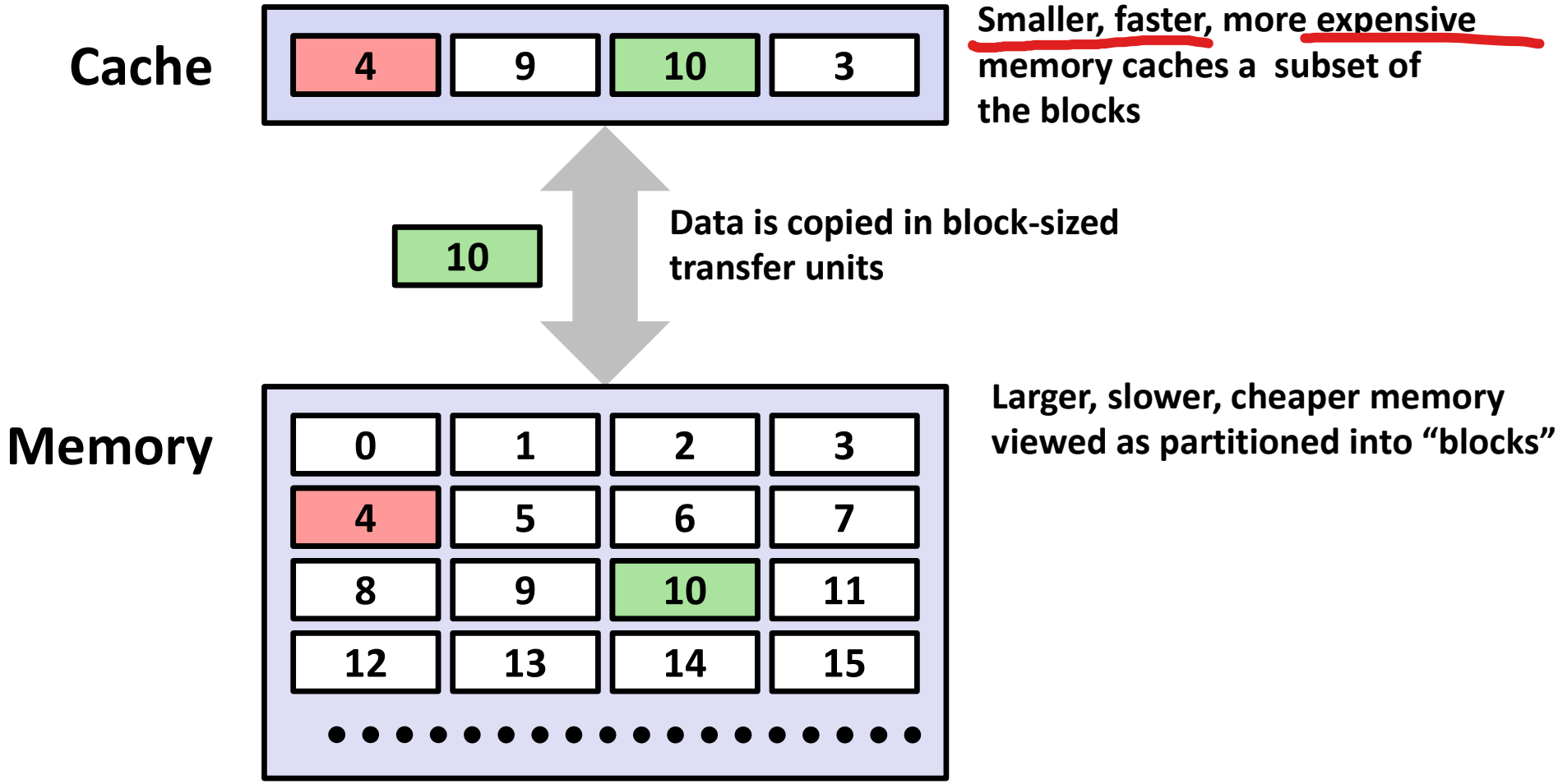
Example Memory Hierarchy



General Cache Concept

Memory에 있는 것들 중 자주 사용되는 것들을 복제해서 가져온 것임. 동일한 데이터 주소를 참조함.

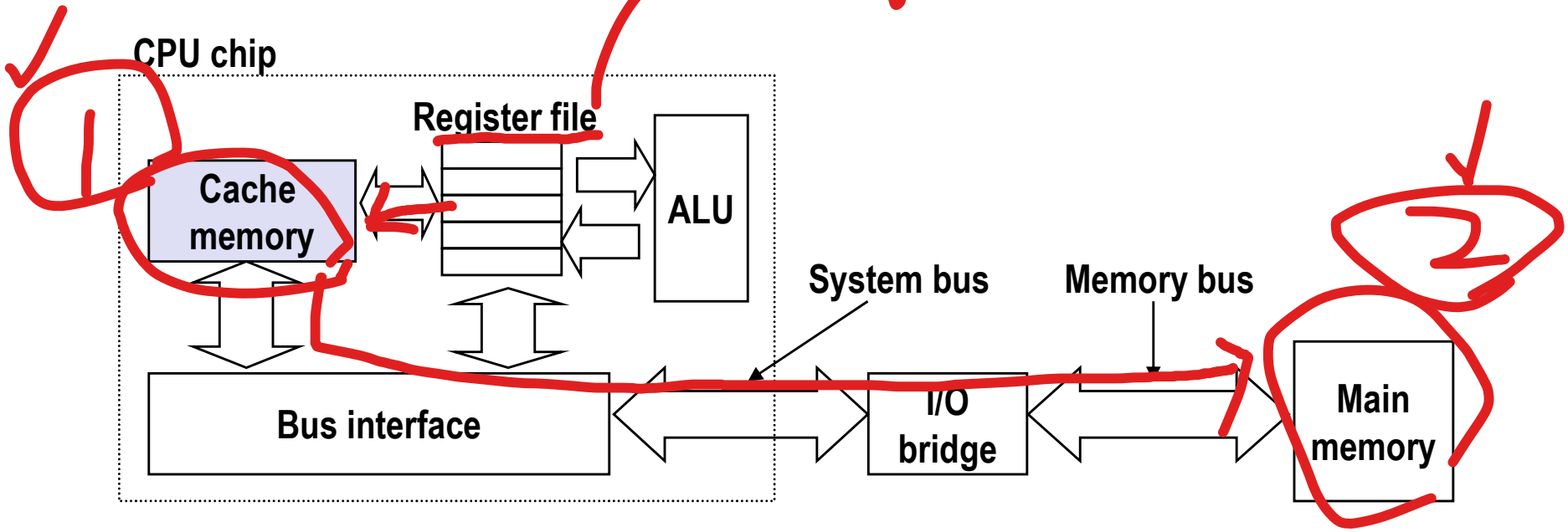
작고 빠르지만,, 비싸다!



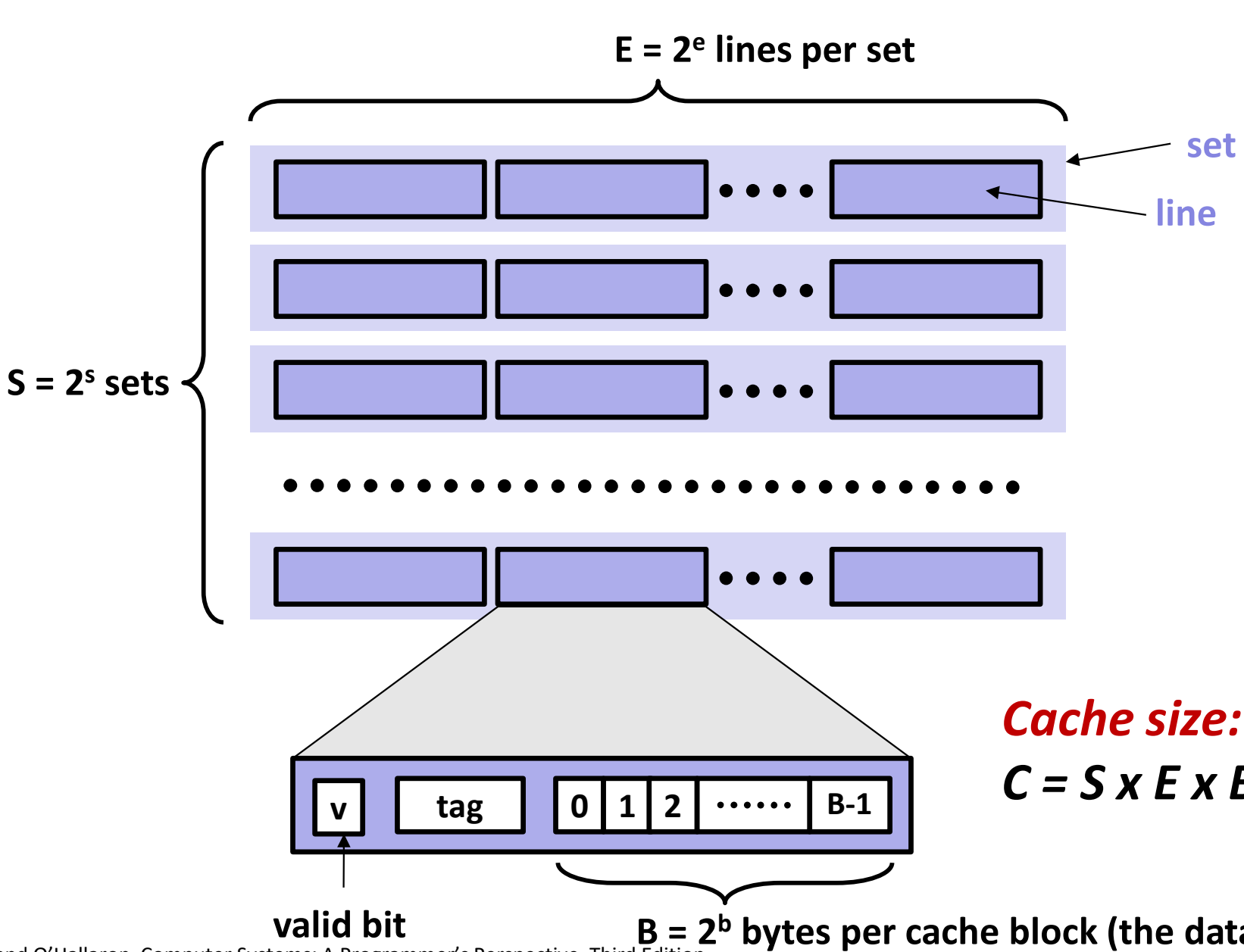
Cache Memories

- **Cache memories are small, fast SRAM-based memories managed automatically in hardware**
  - Hold frequently accessed blocks of main memory
- **CPU looks first for data in cache**
- **Typical system structure:**

[Register file]
특수 목적 Register
- PC, IR..
범용 Register
- EAX, ECX..



General Cache Organization (S, E, B)

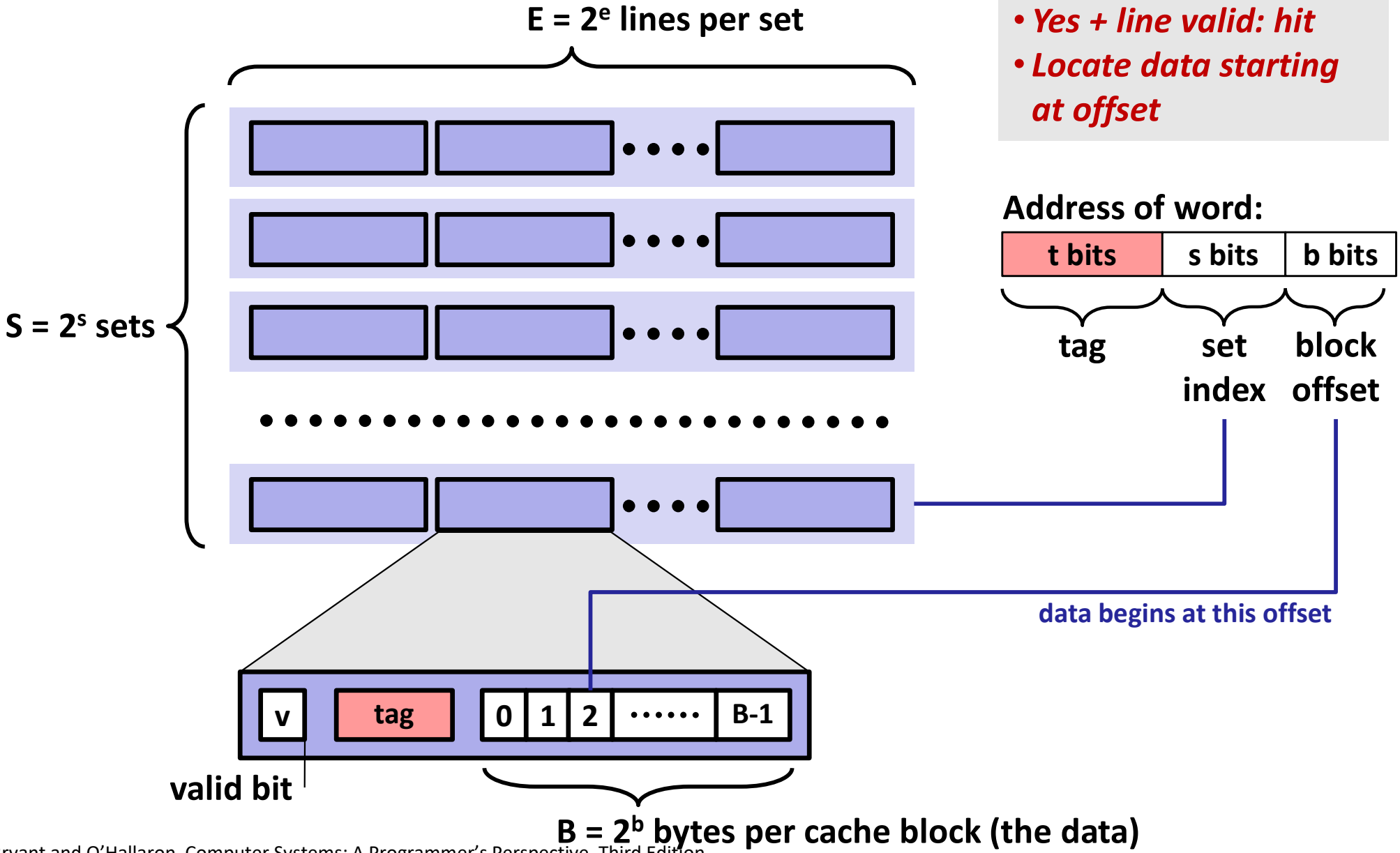


Cache size:

C = S x E x B data bytes

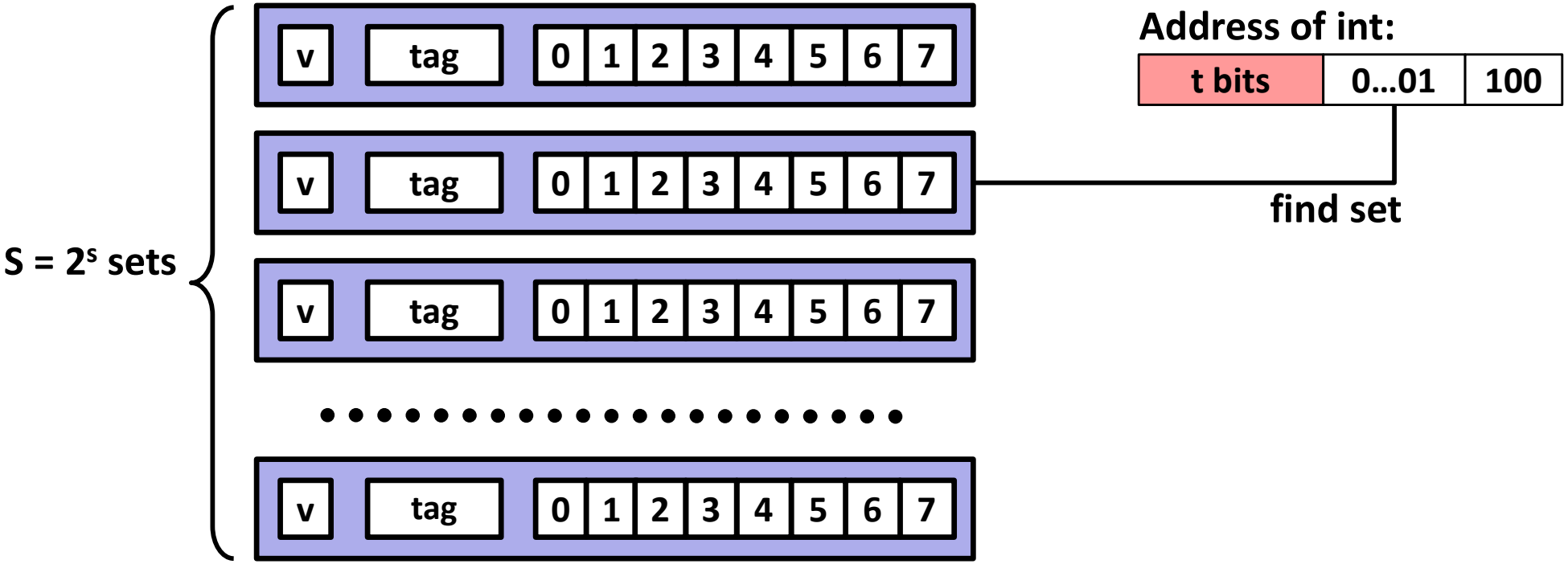
Cache Read

- *Locate set*
- *Check if any line in set has matching tag*
- *Yes + line valid: hit*
- *Locate data starting at offset*



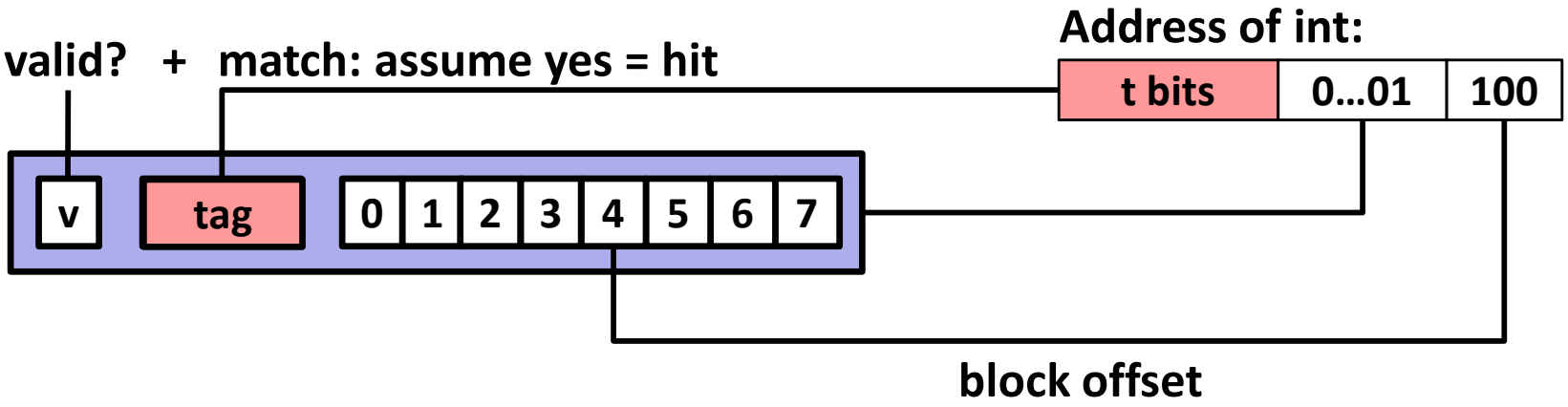
Example: Direct Mapped Cache (E = 1)

Direct mapped: One line per set
Assume: cache block size 8 bytes



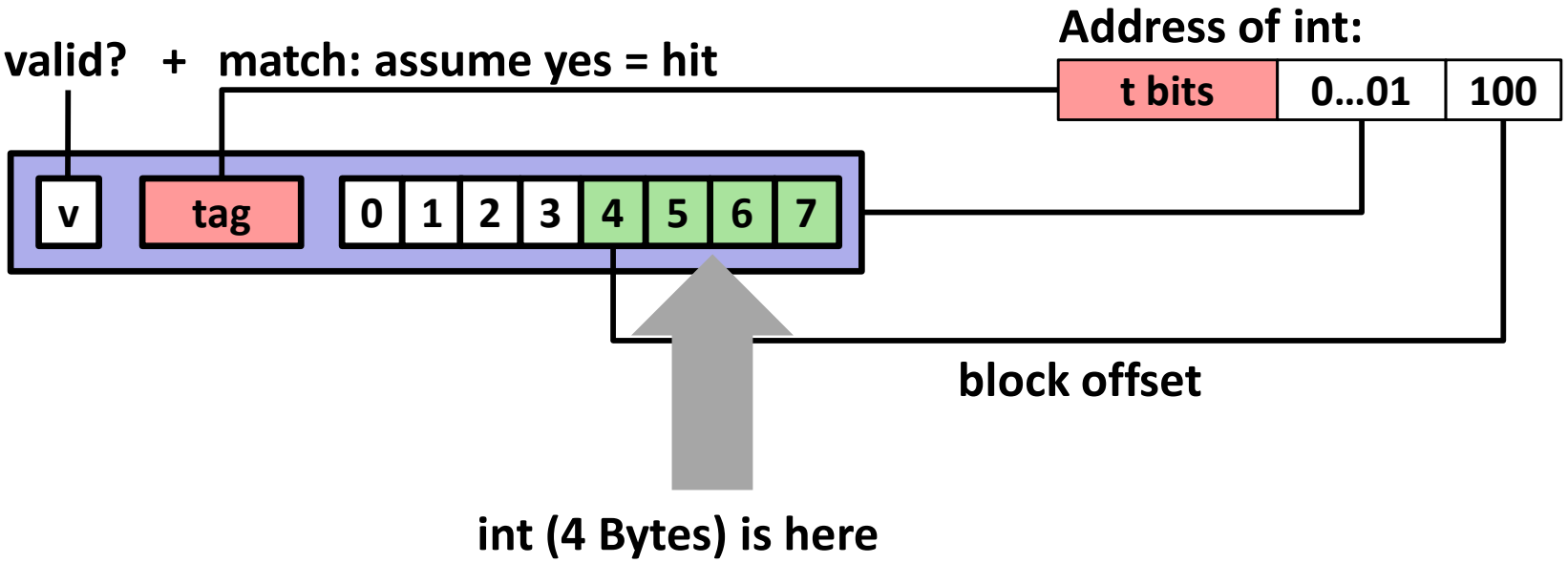
Example: Direct Mapped Cache (E = 1)

Direct mapped: One line per set
Assume: cache block size 8 bytes



Example: Direct Mapped Cache (E = 1)

Direct mapped: One line per set
Assume: cache block size 8 bytes



If tag doesn't match: old line is evicted and replaced

Direct-Mapped Cache Simulation

t=1	s=2	b=1
x	xx	x

M=16 bytes (4-bit addresses), B=2 bytes/block,
S=4 sets, E=1 Blocks/set

Address trace (reads, one byte per read):

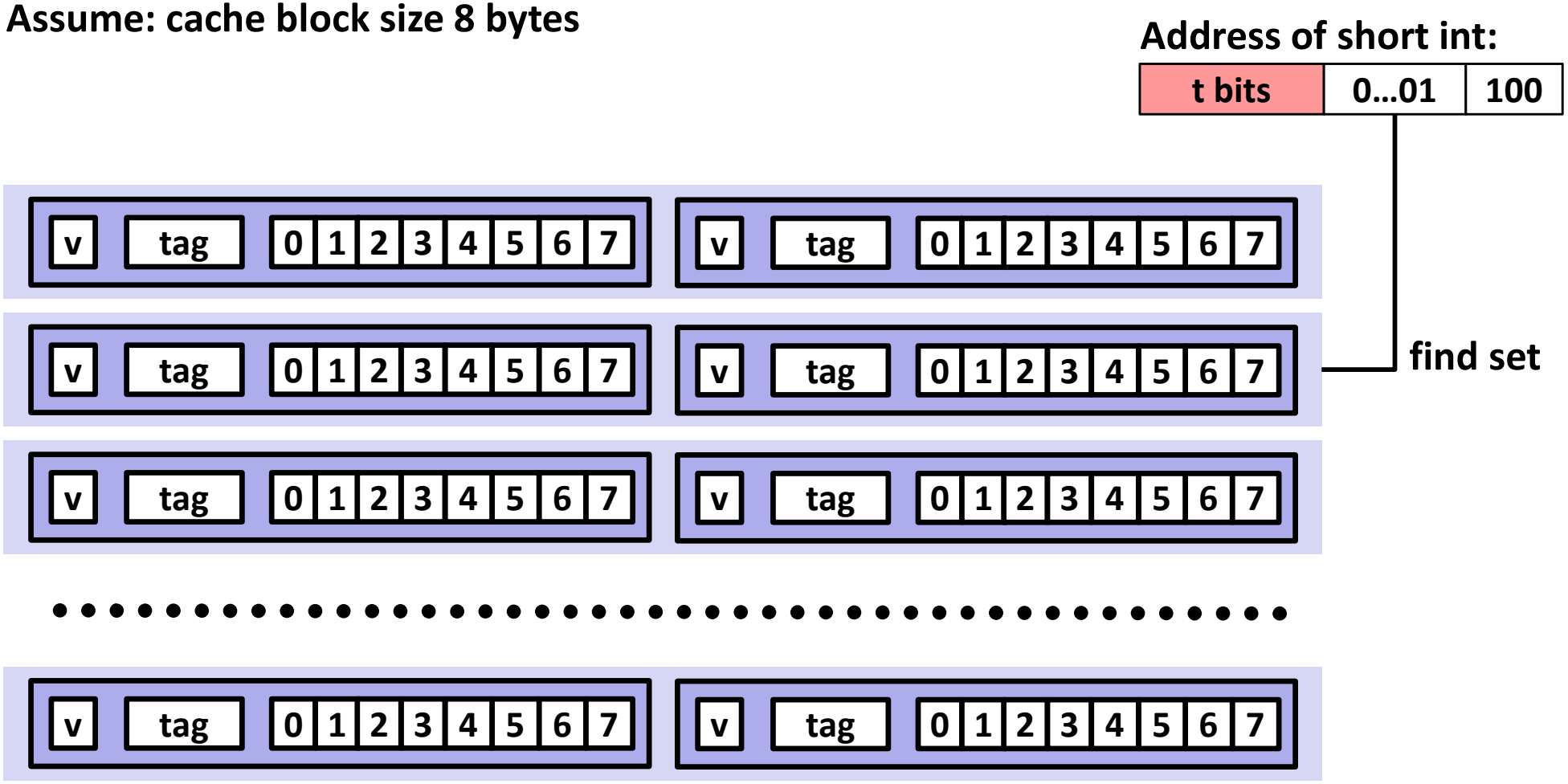
0	**[0$\underline{00}$0$_2$],**	miss
1	**[0$\underline{00}$1$_2$],**	hit
7	**[0$\underline{11}$1$_2$],**	miss
8	**[1$\underline{00}$0$_2$],**	miss
0	**[0$\underline{00}$0$_2$]**	miss

	v	Tag	Block
Set 0	1	0	M[0-1]
Set 1			
Set 2			
Set 3	1	0	M[6-7]

E-way Set Associative Cache (Here: E = 2)

E = 2: Two lines per set

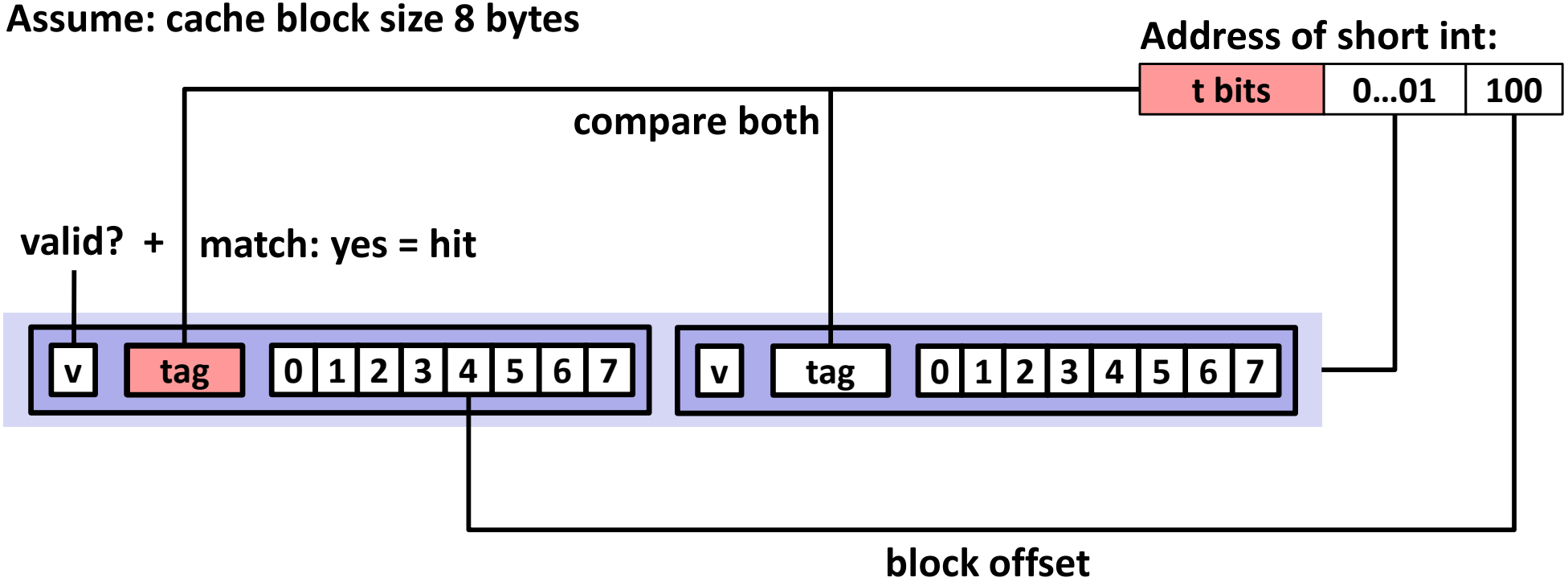
Assume: cache block size 8 bytes



E-way Set Associative Cache (Here: E = 2)

E = 2: Two lines per set

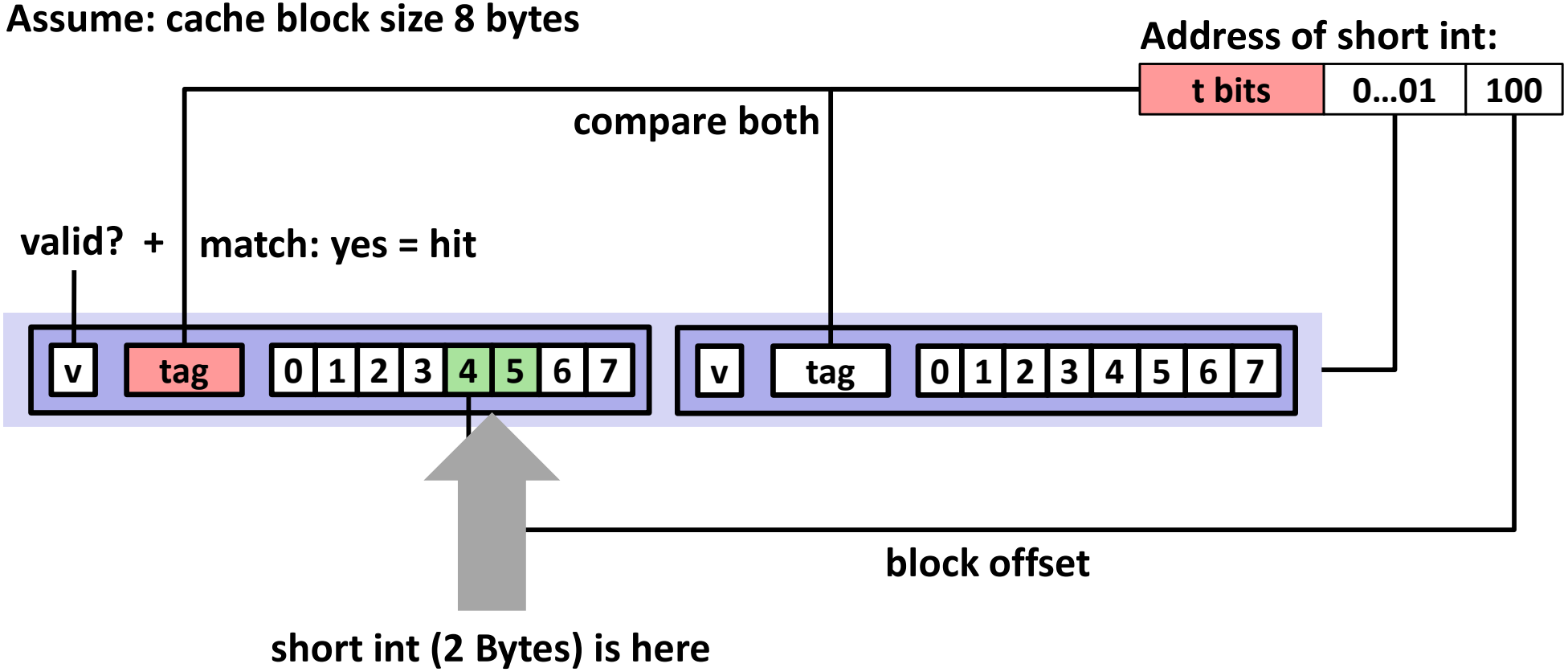
Assume: cache block size 8 bytes



E-way Set Associative Cache (Here: E = 2)

E = 2: Two lines per set

Assume: cache block size 8 bytes



No match:

- One line in set is selected for eviction and replacement
- Replacement policies: random, least recently used (LRU), …

2-Way Set Associative Cache Simulation

t=2	s=1	b=1
xx	x	x

M=16 byte addresses, B=2 bytes/block,
S=2 sets, E=2 blocks/set

Address trace (reads, one byte per read):

0	**[00$\underline{0}$0$_2$],**	miss
1	**[00$\underline{0}$1$_2$],**	hit
7	**[01$\underline{1}$1$_2$],**	miss
8	**[10$\underline{0}$0$_2$],**	miss
0	**[00$\underline{0}$0$_2$]**	hit

	v	Tag	Block
Set 0	1	00	M[0-1]
	1	10	M[8-9]
Set 1	1	01	M[6-7]
	0		

What about writes?

- **Multiple copies of data exist:**
  - L1, L2, L3, Main Memory, Disk
- **What to do on a write-hit?**
  - Write-through (write immediately to memory)
  - Write-back (defer write to memory until replacement of line)
    - Need a dirty bit (line different from memory or not)
- **What to do on a write-miss?**
  - Write-allocate (load into cache, update line in cache)
    - Good if more writes to the location follow
  - No-write-allocate (writes straight to memory, does not load into cache)
- **Typical**
  - Write-through + No-write-allocate
  - **Write-back + Write-allocate**

Write-through :
- 캐시, 메모리를 동시에 업데이트 함.

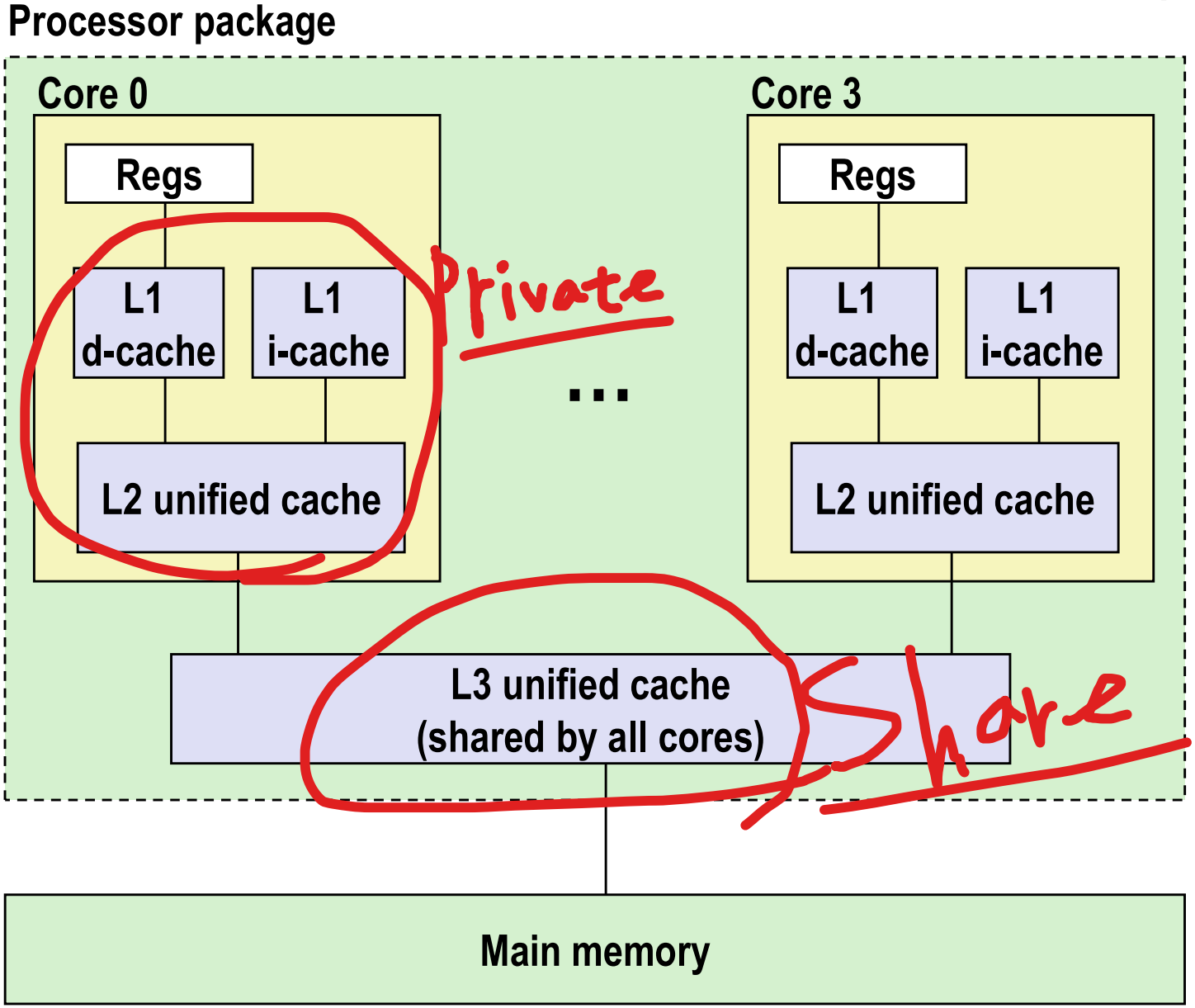
Write-back :
- 캐시만 업데이트 함. (동기화 이슈)
- dirty라고 표시해뒀다가 캐시에서 버릴 때 메모리를 업데이트 함.

Write-allocate :
- 쓰려는 데이터가 캐시에 없으면, 메모리에서 캐시로 먼저 읽어온 뒤 데이터 수정

No-write-allocate :
- 쓰려는 데이터가 캐시에 없으면, 캐시를 건너뛰고 메인 메모리에 직접 데이터 수정

Intel Core i7 Cache Hierarchy

*L1, L2캐시는 Privacy캐시로 각 CPU 코어에서만 사용된다.
*L3캐시는 Share캐시로 여러 CPU 코어에서 필요한 만큼 사용한다.



L1 i-cache and d-cache:
32 KB, 8-way,
Access: 4 cycles

L2 unified cache:
256 KB, 8-way,
Access: 10 cycles

L3 unified cache:
8 MB, 16-way,
- Access: 40-75 cycles

Block size: 64 bytes for all caches.

Cache Performance Metrics

- **Miss Rate**
  - Fraction of memory references not found in cache (misses / accesses) = 1 – hit rate
  - Typical numbers (in percentages):
    - 3-10% for L1
    - can be quite small (e.g., < 1%) for L2, depending on size, etc.
- **Hit Time**
  - Time to deliver a line in the cache to the processor
    - includes time to determine whether the line is in the cache
  - Typical numbers:
    - 4 clock cycle for L1
    - 10 clock cycles for L2
- **Miss Penalty**
  - Additional time required because of a miss
    - typically 50-200 cycles for main memory (Trend: increasing!)

Let's think about those numbers

- **Huge difference between a hit and a miss**
  - Could be 100x, if just L1 and main memory

- **Would you believe 99% hits is twice as good as 97%?**
  - Consider:
    cache hit time of 1 cycle
    miss penalty of 100 cycles

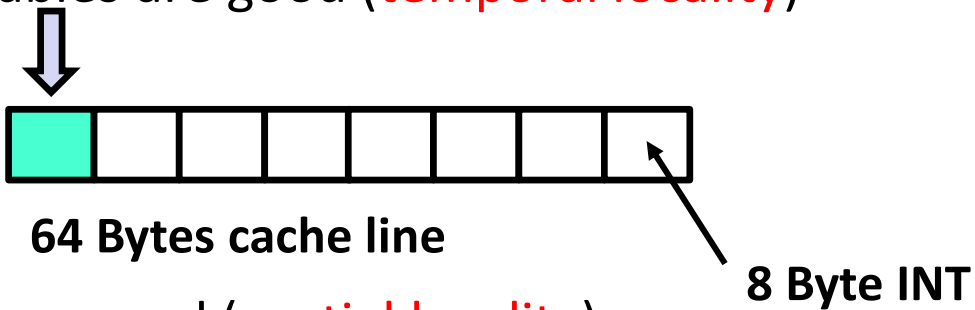
  - Average access time:
    97% hits: 1 cycle + 0.03 * 100 cycles = **4 cycles**
    99% hits: 1 cycle + 0.01 * 100 cycles = **2 cycles**

- **This is why "miss rate" is used instead of "hit rate"**

Writing Cache Friendly Code

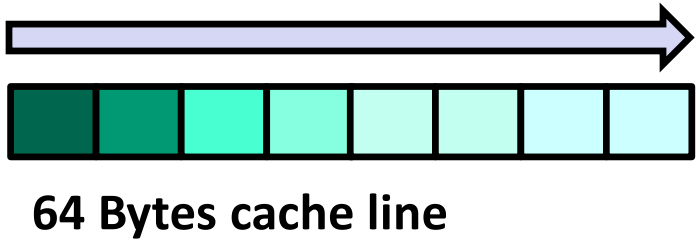
- **Make the common case go fast**
  - Focus on the inner loops of the core functions
- **Minimize the misses in the inner loops**
  - Repeated references to variables are good (temporal locality)

```
for (int i = 0; i < 10,000; i++)
  sum += A[128];
```



  - Stride-1 reference patterns are good (spatial locality)

```
for (int i = 0; i < N; i++)
  sum += A[i];
```



Key idea: Our qualitative notion of locality is quantified through our understanding of cache memories

Memory Hierarchy: Summary

- **Storage Technologies and Trends**
  - Gap widening between CPU speed and memory/storage latency
- **Locality of Reference**
  - Temporal locality and Spatial locality
- **Caching in the Memory Hierarchy**
  - Multi-level caches (L1, L2, L3) reduce memory access latency
  - Cache block (line), Replacement policies
- **Cache Memory Organization and Operation**
  - Direct-mapped: 1 line per index (fastest, conflict-prone)
  - Set-associative: multiple lines per set (balanced)
  - Fully associative: any line for any block (flexible, slow)
  - Tag, Index, Offset structure used to identify memory addresses
- **Performance Impact of Caches**
  - Improving cache hit rate is key to CPU performance